

ACF Blocks: Repeater Rendering & Gutenberg Preview Patterns

A practical reference for PHP-rendered ACF blocks with repeaters, editor previews, and block-inserter examples.

This guide consolidates the recommended approach for keeping frontend rendering clean while still giving editors useful visual feedback in both Gutenberg and the block inserter.

Core idea: Treat ACF as the data-entry layer, normalise its output into ordinary PHP arrays, and let one renderer handle real content and temporary preview content.

1. Recommended architecture

- `block.json` owns block registration, assets, and inserter preview configuration.
- ACF owns the content fields, including Repeater fields.
- `render.php` retrieves the entire repeater as an array and renders it with `foreach`.
- If the repeater is empty and the block is being previewed in Gutenberg, inject temporary placeholder rows only when appropriate.
- Use `block.json` example metadata to trigger the visual block-inserter preview.
- Render the same HTML and CSS in Gutenberg and on the frontend wherever possible.

The important change is not simply `have_rows()` versus `foreach`. It is separating data acquisition from rendering. Once the repeater has been normalised into a predictable array, the template no longer needs to care whether a row came from saved ACF data or from temporary preview fixtures.

Suggested block structure

```
blocks/  
└─ featured-post-grid-four/  
   │ block.json  
   │ render.php  
   │ style.scss  
   │ editor.scss      // only if Gutenberg needs overrides  
   └─ script.js       // only if the block is interactive
```

2. `block.json` for a modern ACF block

For new work, a `block.json`-based registration keeps metadata, assets, preview configuration, and ACF block settings in one place.

```
{
  "$schema": "https://schemas.wp.org/trunk/block.json",
  "apiVersion": 3,
  "name": "acf/featured-post-grid-four",
  "title": "Featured Post Grid - Four",
  "description": "Displays a four-column featured post grid.",
  "category": "design",
  "icon": "grid-view",

  "example": {
    "viewportWidth": 1200,
    "attributes": {
      "mode": "preview",
      "data": {
        "is_example_preview": true
      }
    }
  },

  "acf": {
    "blockVersion": 3,
    "renderTemplate": "render.php",
    "hideFieldsInSidebar": true,
    "expandedEditorButtonText": "Edit cards"
  },

  "supports": {
    "align": false,
    "anchor": true
  },

  "style": "file:./style.css"
}
```

For repeater-heavy components, the expanded ACF editor is generally a better editing surface than trying to fit a long repeater into Gutenberg's narrow sidebar. The canvas can remain a visual preview while the editor opens the larger field interface when the user wants to edit the rows.

3. Normalise repeater data before rendering

Instead of driving the template through repeated ACF sub-field calls:

```
while (have_rows('custom_pivot_post')) : the_row();
    $image = get_sub_field('image');
    $heading = get_sub_field('heading');
    ...
endwhile;
```

Retrieve the repeater once and treat it as ordinary PHP data:

```
$post_rows = get_field('custom_pivot_post');
$post_rows = is_array($post_rows) ? $post_rows : [];
```

Define one placeholder row:

```
$placeholder_card = [
    'add_post_link' => '',
    'image'         => [],
    'heading'       => 'Example card heading',
    'post_desc'     => 'Add a short description for this card.',
];
```

If you need four identical placeholders:

```
$post_rows = array_fill(0, 4, $placeholder_card);
```

Why this scales: If you later add category, author, date, eyebrow, button text, theme, icon, or other fields, you extend the row shape once and continue rendering through the same foreach loop.

4. Design for three distinct render states

State	Saved data	Temporary data	Goal
Frontend	Use real ACF rows	None	Render only real content
Gutenberg editor	Use real rows when present	Optional empty-state guidance	Show what is actually configured
Block inserter preview	Usually none	Representative example content	Show what the block can do

This distinction becomes important because an empty inserter preview is often a poor advertisement for a component, while an actual newly inserted block should not silently enable optional features.

5. Use a custom flag for the block-inserter example

The built-in preview state alone is too broad when you want the block inserter to look richer than a newly inserted block. A custom value in `example.attributes.data` gives the renderer an explicit signal that it is producing the example preview.

```
$is_example_preview = !empty($block['data']['is_example_preview']);
$is_editor_preview  = $is_preview && !$is_example_preview;
$is_frontend        = !$is_preview;
```

Useful mental model: Inserter = show what this block can do. Editor = show what the user has actually configured. Frontend = render only saved content.

6. Example: show two optional headings in the inserter, but keep them disabled by default

Real ACF defaults can remain false:

```
$show_heading_1 = get_field('show_heading_1') ?: false;
$heading_1      = get_field('heading_1') ?: '';

$show_heading_2 = get_field('show_heading_2') ?: false;
$heading_2      = get_field('heading_2') ?: '';
```

Override only the temporary inserter example:

```
if ($is_example_preview) {
    $show_heading_1 = true;
    $heading_1 = 'Optional primary heading';

    $show_heading_2 = true;
    $heading_2 = 'Optional secondary heading';
}
```

The renderer itself stays ordinary:

```
<?php if ($show_heading_1) : ?>
    <h2><?php echo esc_html($heading_1); ?></h2>
<?php endif; ?>

<?php if ($show_heading_2) : ?>
    <h3><?php echo esc_html($heading_2); ?></h3>
<?php endif; ?>
```

The important point is that these demonstration values are temporary. They make the picker preview informative without changing the actual starting state of a newly inserted block.

7. Keep product defaults separate from demo state

Avoid setting optional controls to true in ACF merely to make the inserter preview look good. That makes the option genuinely enabled when the block is inserted.

Product default

What should actually happen when the editor inserts the block.

Demo state

What best communicates the component's capabilities in the block library.

Recommended rule

Keep real defaults truthful; override values only when `is_example_preview` is true.

8. Reusable render.php pattern

```
<?php

$post_rows = get_field('custom_pivot_post');
$post_rows = is_array($post_rows) ? $post_rows : [];

$bg_colour = get_field('bg_colour_copy2') ?: '';

$temp_image = get_template_directory_uri()
    . '/blocks/global/featured-post-grid-four/image-sample.jpg';

$placeholder_card = [
    'add_post_link' => '',
    'image'         => [],
    'heading'        => 'Example card heading',
    'post_desc'      => 'Add a short description for this card.',
];

$is_example_preview = !empty($block['data']['is_example_preview']);
$is_editor_preview  = $is_preview && !$is_example_preview;

if (empty($post_rows) && $is_example_preview) {
    $post_rows = array_fill(0, 4, $placeholder_card);
}

/*
 * Optional: if you also want a lightweight empty-state
 * visual inside Gutenberg, populate a smaller fixture here.
 */
if (empty($post_rows) && $is_editor_preview) {
    // Example:
    // $post_rows = array_fill(0, 1, $placeholder_card);
}

if (empty($post_rows)) {
    return;
}

?>
```

Then render all rows through one loop:

```

<?php foreach ($post_rows as $post) :

    $image = is_array($post['image'] ?? null)
        ? $post['image']
        : [];

    $link_url = $post['add_post_link'] ?? '';

    $url      = $image['url'] ?? $temp_image;
    $alt      = $image['alt'] ?? '';
    $width    = $image['width'] ?? 676;
    $height   = $image['height'] ?? 416;

    $heading = $post['heading']
        ?? 'Example card heading';

    $description = $post['post_desc']
        ?? 'Add a short description for this card.';
?>

<article class="card">
    ...
</article>

<?php endforeach; ?>

```

9. Use the real component CSS in Gutenberg

Prefer one component stylesheet shared by the frontend and editor. Add editor-only CSS only when Gutenberg needs a specific override.

```

"style": "file:./style.css",
"editorStyle": "file:./editor.css" // only if needed

```

For example, editor-only CSS might disable pointer events on links so a preview remains easy to select without navigating away.

10. Interactive blocks need editor re-initialisation

Tabs, accordions, sliders, and similar blocks need extra care because ACF can replace the preview HTML after a field change. Event listeners attached to the old DOM disappear with it.

```
function initBlock(block) {
    // initialise tabs, slider, accordion, etc.
}

document.querySelectorAll('.my-block').forEach(initBlock);

if (window.acf) {
    window.acf.addAction(
        'render_block_preview/type=my-block',
        ($block) => {
            initBlock($block[0]);
        }
    );
}
```

11. Practical decision guide

Situation	Recommended approach
Simple fixed fields	Direct <code>get_field()</code> values are fine.
Repeater / repeated cards	Get the repeater once as an array, normalise it, then <code>foreach</code> .
Empty block in editor	Use lightweight guidance or minimal placeholders only if useful.
Insert preview	Use example metadata plus an <code>is_example_preview</code> flag and richer fixtures.
Optional features	Keep them disabled as real defaults; enable them only for the example preview.
Complex repeaters	Prefer ACF's expanded editing UI rather than cramming fields into the sidebar.
Interactive block	Re-initialise JS after ACF re-renders the block preview.

12. The pattern to standardise across a block library

```
ACF fields
↓
get_field('repeater')
↓
normal PHP array
↓
determine render state
├─ inserter example → rich temporary fixture
├─ Gutenberg editor → real data / light empty state
├─ frontend → saved data only
↓
one foreach renderer
↓
same HTML + same component CSS
```

Bottom line: Do not make the frontend template itself a maze of ACF calls. Build predictable data first, decide which render state you are in, then pass everything through one renderer.

References

- Advanced Custom Fields - ACF Block Configuration via block.json: <https://www.advancedcustomfields.com/resources/acf-block-configuration-via-block-json/>
- Advanced Custom Fields - ACF Blocks v3: <https://www.advancedcustomfields.com/resources/acf-blocks-v3/>
- Advanced Custom Fields - Placeholder content in custom ACF blocks: <https://www.advancedcustomfields.com/resources/set-placeholder-content-in-custom-acf-blocks/>
- Advanced Custom Fields - ACF register block type / example data: https://www.advancedcustomfields.com/resources/acf_register_block_type/
- Advanced Custom Fields - Enabling inline editing: <https://www.advancedcustomfields.com/resources/enabling-inline-editing/>
- WordPress Developer Resources - Block metadata: <https://developer.wordpress.org/block-editor/reference-guides/block-api/block-metadata/>
- WordPress Developer Resources - Block registration / example preview: <https://developer.wordpress.org/block-editor/reference-guides/block-api/block-registration/>